RAG 与本体知识平台技术架构书

1　项目定位

本平台面向企业知识管理与智能问答，将文档解析、检索增强生成、领域本体建模和结构化知识抽取组织为相互协作的服务体系。平台既可以从原始资料中找到与问题相关的内容，也可以把资料中的业务对象、属性和关系沉淀为可复用的知识。

总体架构采用应用接入层、知识处理层、模型能力层与数据基础设施分工。RAG 链路承担文档入库与问答证据召回；ontology-rag 侧承担领域本体生成、承担本体管理、实例抽取、图谱组织与结构化检索。

1.1　主要业务能力

能力	解决的问题
企业知识问答	从文档、FAQ 和补充知识中召回证据，为模型回答提供依据。
领域知识建模	从业务资料中归纳对象类型、属性与关系，形成统一语义结构。
结构化知识提取	按照本体抽取实例和事实，支持关系查询、超图展示及事实检索。

1.2　建设成果与范围

建设成果包括文档入库与问答能力、标签或对象类型抽取流程、本体创建与实例抽取流程，以及支持部署验收、安全控制和日常运行的机制。范围覆盖 Chatbot 应用服务、rag-server、rag-loader、rag-magicpdf、rag-embedding、

embedding_service、rag-rerank、maas-ontology、rag-ontology 等微服务和配套基础设施。联网搜索与实时问答作为按需启用的补充知识源。

“标签抽取”采用业务侧的操作名称，对应 maas-ontology 微服务的类或对象类型抽取能力，与 RAG 文档关键词、摘要标签的生成属于不同流程。后续是否执行属性抽取，由领域建模场景决定；实例抽取对应 rag-ontology 微服务则以已创建的本体为前提。

2　业务问题与建设目标

2.1　业务问题

企业资料通常分散在不同文件和业务入口中，存在格式不统一、扫描内容难检索、人工整理耗时和问答缺少依据等问题。仅通过关键词搜索，难以直接回答同义表达或跨段落问题；仅让模型阅读资料，又容易缺少可重复的处理流程和结果控制。

在领域知识应用中，还需要区分对象类型与具体实例。例如，“产品”“套餐”属于业务类型，某项具体产品及其属性属于实例。若没有先定义对象和属性，文档抽取结果容易出现字段不一致、关系方向不明确以及业务展示难以复用的问题。

2.2　目标与业务交付

业务问题	建设目标	业务侧可见成果
资料难以统一使用	建立可追踪的解析、切分与索引过程。	上传后可查看处理进度和可检索状态。
查找与答复效率不足	通过召回、重排序和模型生成组织答案。	问题获得相关回答，并能查看关联资料。
领域定义依赖人工整理	支持标准资料驱动的类型与属性抽取。	形成可校对、可维护的业务对象定义。

业务问题	建设目标	业务侧可见成果
抽取结果结构不统一	以本体约束实例抽取和关系表达。	获得结构化实例、属性与关系展示。
处理过程难以运行维护	统一业务状态、失败处理和运行监控。	任务可定位、可重试，故障有明确责任。

2.3 业务约束

所有流程以资料可访问、配置可用和用户有权限为前提。上传成功不等于索引完成，本体任务提交成功不等于本体可用于实例抽取。系统应以阶段结果判断可用性，并把失败原因转换为业务可理解的提示。

3　总体逻辑架构

图 1 从职责角度组织能力。应用接入层处理用户请求和业务编排；知识处理层执行文档与本体任务；模型能力提供 OCR、嵌入、重排序和生成；数据基础设施保存业务记录、文件与索引。

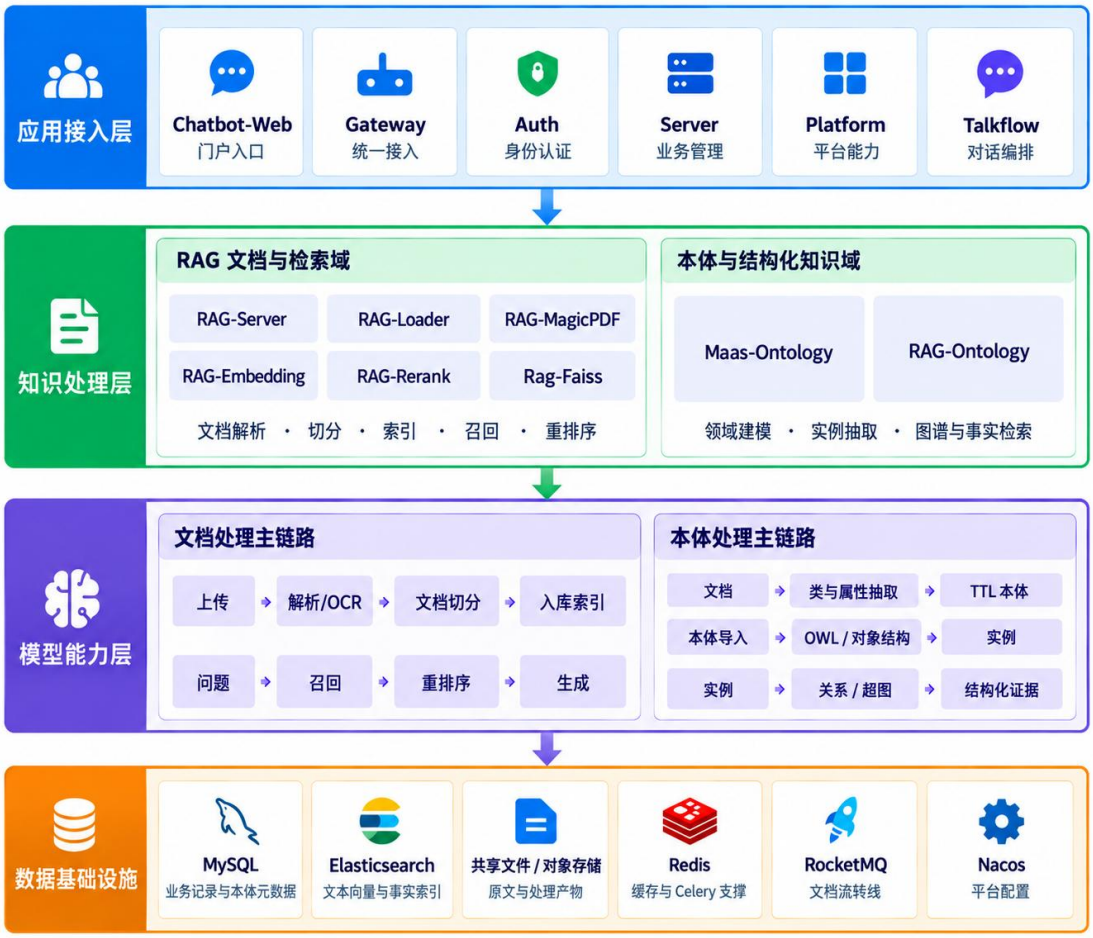


图 1　RAG 与本体知识平台逻辑架构图

如图 1 所示，RAG 与本体知识平台采用分层解耦的总体逻辑架构，自上而下划分为应用接入层、知识处理层、模型能力层和数据基础设施层。各层通过标准化接口协同工作，形成从用户请求接入、知识加工与检索，到模型推理和数据持久化的完整闭环。该架构兼顾文档型非结构化知识处理与本体驱动的结构化知

识管理，可支撑智能问答、知识检索、文档解析、领域建模及业务对话编排等应用场景。

应用接入层面向最终用户、业务系统及运维管理人员提供统一入口。其中，Chatbot-Web 用于承载面向用户的问答与知识服务交互；Gateway 负责服务统一接入、请求路由和接口治理；Auth 提供身份认证与访问控制；Server 和 Platform 分别承担业务管理、平台配置及能力运营等职责；Talkflow 则负责对话流程的组织、编排和执行。该层屏蔽底层知识处理与模型调用的技术细节，使上层业务能够以统一方式调用平台能力。

知识处理层是平台知识构建与知识服务的核心，主要包括“RAG 文档与检索域”和“本体与结构化知识域”两部分。RAG 文档与检索域通过 RAG-Loader、RAG-MagicPDF、RAG-Server、RAG-Embedding、RAG-Rerank 和 RAG-Faiss 等组件，实现文档导入、解析、切分、向量化、索引构建、召回和重排序等处理流程，将原始文档转化为可被大模型高效利用的知识片段。本体与结构化知识域通过 Maas-Ontology、RAG-Ontology 等组件开展领域建模、实体与属性抽取、图谱构建以及事实检索，将文档中的关键业务概念、实体关系和规则沉淀为结构化知识资产。

模型能力层为知识处理过程提供通用智能能力支撑，并分别形成文档处理主链路和本体处理主链路。在文档处理链路中，系统完成文件上传、文档解析或 OCR、内容切分、入库索引等离线加工，并在用户提问时依次执行知识召回、结果重排序和大模型生成，从而输出有依据的问答结果。在本体处理链路中，系统从原始文档中抽取实体、属性和关系，构建 TTL 本体或 OWL/对象结构，并逐步形成实例、关系及结构化证据，为语义检索、知识推理和精准问答提供支持。

数据基础设施层为平台稳定运行提供数据存储、缓存、消息通信和配置管理能力。MySQL 用于保存业务数据、用户数据及本体元数据等关系型记录；Elasticsearch 支撑文本内容和事实信息的检索；共享文件或对象存储保存原始文件、解析结果及中间产物；Redis 提供缓存与 Celery 等异步任务支持；RocketMQ 用于文档处理、索引构建等异步任务的消息传递；Nacos 则承担服务注册、配置管理和运行环境治理职责。通过多类型数据组件协同，平台能够满足大规模文档处理、低延迟检索和高并发服务的需求。

4　总体业务流程与核心服务流程

如图 2 所示，平台将用户访问、业务编排、文档知识处理、模型推理和本体知识构建统一纳入整体服务调用流程。用户首先通过 Chatbot-Web 访问系统，经过 Chatbot-auth 完成登录认证后，由 Chatbot-gateway 统一接收并路由请求。网关根据请求类型，将业务请求分发至 Chatbot-server、Chatbot-talkflow 或相关知识服务，实现业务管理、对话流程控制和智能问答能力调用。

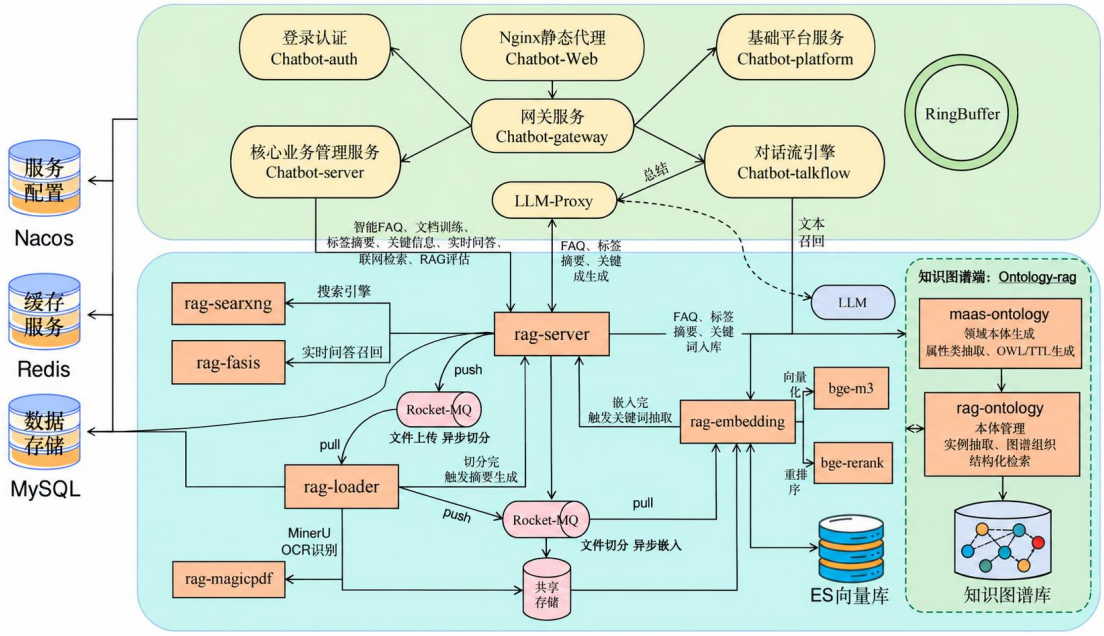


图 2　RAG 与本体知识平台业务流程图

在**智能问答流程**中，Chatbot-talkflow 负责组织具体对话步骤，并通过 LLM-Proxy 服务调用大模型能力。对于需要知识增强的提问，系统将问题发送至 rag-server 服务，由其统一协调检索、召回、重排序和生成等过程。rag-server 服务可调用 rag-searxng 服务执行关键词或全文检索，调用 rag-faiss 服务完成向量相似度召回，并结合 bge-rerank 模型对候选知识片段进行重排序；必要时通过

LLM 完成答案生成、摘要提取、关键词抽取和标签生成，最终将基于知识依据的回答返回至上层对话服务。

在**文档入库与知识加工流程**中，业务人员上传文件后，rag-server 服务将文件处理任务写入 RocketMQ，由 rag-loader 服务异步拉取任务并执行文档解析、内容切分及处理状态回传。对于扫描件、图片或复杂版式文档，rag-loader 服务可调用 rag-magicpdf 服务和 MinerU OCR 模型能力进行文本识别和结构化解析。处理完成后的原始文件及中间结果存入共享存储，切分得到的文本片段则进入后续向量化和索引构建流程。

在**向量化与检索索引流程**中，rag-embedding 服务从消息队列获取待处理的文本片段，调用 bge-m3 模型生成文本向量，并将向量及关联元数据写入 ES 向量库。系统同时可利用 Elasticsearch 支撑文本检索、标签检索和事实查询，使关键词检索与语义向量检索能够结合使用，为实时问答、相似内容匹配和知识检索提供基础支撑。

在**本体知识处理流程**中，文档内容、问答过程中的标签和摘要等信息可同步进入知识图谱侧。maas-ontology 服务负责领域本体建模、实体与属性抽取以及 OWL/TTL 本体生成；rag-ontology 服务进一步完成本体管理、实例构建、关系组织和结构化检索，并将形成的知识网络保存至知识图谱库。本体侧可为检索增强问答提供实体关联、事实检索和语义约束能力；其具体调用顺序和编排方式可结合实际业务场景进行配置。

此外，Nacos 为各微服务提供配置管理和服务发现能力，Redis 用于缓存热点数据和提升访问性能，MySQL 用于保存业务记录、任务状态及相关元数据信息，RocketMQ 则负责文档上传、切分、嵌入等耗时任务的异步解耦。通过上述

服务协同，平台形成了“请求接入—知识检索—模型生成—文档加工—本体沉淀—数据存储”的完整业务闭环。

4.1 标签与对象类型抽取流程

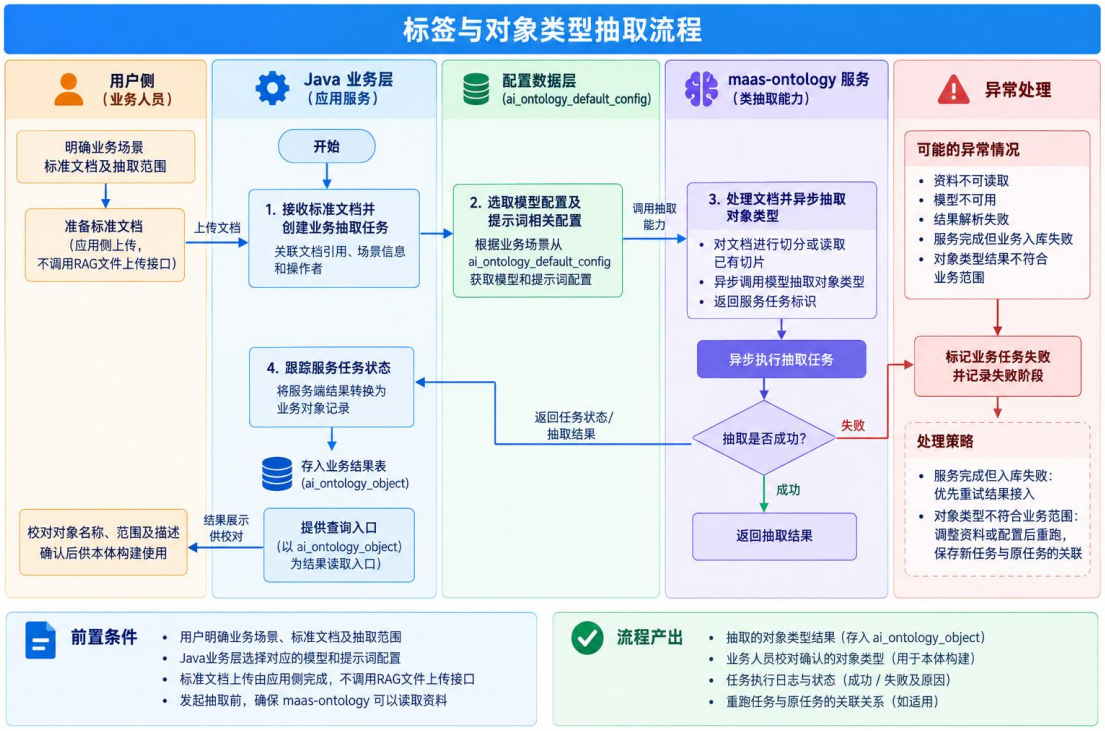


图 3　标签与对象类型抽取流程图

图 3 标签与对象类型抽取流程图展示了标准资料从业务侧提交、配置选择、模型抽取到结果入库和人工校对的完整链路。Java 业务层负责创建抽取任务并读取模型、提示词配置，maas-ontology 服务异步执行文档读取、切分及对象类型抽取；抽取完成后，结果回写为本体类对象记录，供业务人员确认对象名称、范围和描述。流程同时区分了服务执行状态与业务入库状态，便于对资料不可读、模型调用失败或结果接入失败等情况进行定位和重试。

4.1.1 流程目的与前置条件

该流程从标准业务资料中提取对象类型，为后续本体定义提供候选内容。用户应先明确业务场景、标准文档及抽取范围，Java 业务层选择对应的模型和提示词配置。按照业务流程，标准文档上传阶段由应用侧完成，不调用 RAG 文件上传接口；在发起抽取前，必须保证 maas-ontology 可以读取资料。

4.1.2 执行顺序与模块协作

第一步，Java 业务层接收标准文档并创建业务抽取任务，将文档引用、场景信息和操作者关联到任务。第二步，从本体配置数据表(ai_ontology_default_config)选取模型配置及提示词相关配置，调用 maas-ontology 服务的类抽取能力。第三步，maas-ontology 服务对文档进行所需的切分或读取已有切片，异步抽取对象类型，返回服务任务标识。第四步，业务层跟踪服务任务状态，将服务端结果转换为业务对象记录。用户侧查询以本体类对象(ai_ontology_object)为结果读取入口。第五步，业务人员校对对象名称、范围及描述，确认后供本体构建使用。

责任模块	输入	输出与完成条件
Java 业务编排	标准文档、场景、模型与提示词配置。	业务任务及服务任务关联建立成功。
maas-ontology	可读取的文档及抽取配置。	对象类型结果生成，服务任务进入完成状态。

责任模块	输入	输出与完成条件
结果接入与展示	服务抽取结果、业务任务标识。	结果保存至业务对象记录，页面可查询。
业务校对	对象类型及描述。	确认对象边界，决定是否继续属性建模。

4.1.3 设计理由与异常处理

配置由业务层管理，使同一场景可以复用明确的模型和提示词；执行由 maas-ontology 服务承担，避免把耗时模型调用放在页面请求中。业务结果与服务结果分别管理，使用户查询不会直接依赖任务产物路径或模型响应格式。

资料不可读取、模型不可用或结果解析失败时，业务任务应标记对应失败阶段。服务完成但业务入库失败时，优先重试结果接入，避免重复调用模型。对象类型结果不符合业务范围时，应调整资料或配置后重跑，保存新任务与原任务的关联。

4.2 本体创建与实例抽取流程

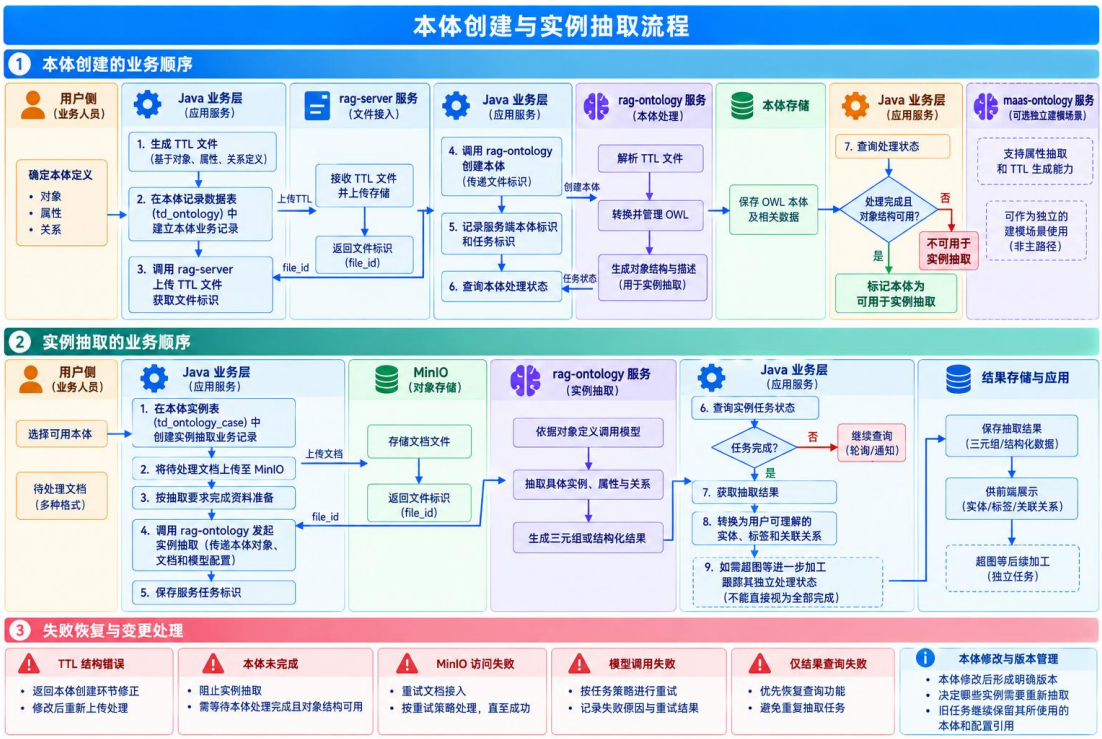


图 4 本体创建与实例抽取流程图

图 4 本体创建与实例抽取流程图分为本体创建、实例抽取及失败恢复三个部分。本体创建阶段由业务层根据已确认的对象、属性和关系生成 TTL 文件，经 rag-server 服务上传后调用 rag-ontology 服务进行解析和 OWL 管理；本体对象结构生成完成后，方可标记为可用于实例抽取。实例抽取阶段以可用本体和待处理文档为输入，通过 rag-ontology 服务提取实体、属性及关系，并将三元组或结构化结果保存后用于页面展示和后续图谱加工。图中同时明确了本体版本管理、TTL 校验、文件访问失败及模型调用异常等控制节点。

4.2.1 创建本体的业务顺序

Java 业务层根据已确定的对象、属性和关系定义生成 TTL 文件，并在本体记录数据表(td_ontology)中建立本体业务记录。TTL 上传通过 rag-server 服务

完成，获得供后续处理引用的文件标识。业务层再调用 rag-ontology 服务创建本体，记录服务端本体标识和任务标识，并查询本体处理状态。

rag-ontology 服务后台任务解析 TTL、转换并管理 OWL，生成用于实例抽取的对象结构与描述。只有相关处理阶段完成、本体对象结构可用后，业务侧才能将本体标记为可用于实例抽取。Java 生成 TTL 是当前业务流程的主路径；maas-ontology 服务支持属性抽取和 TTL 生成的能力，可作为独立建模场景使用。

4.2.2 实例抽取的业务顺序

业务层选择可用本体，在本体实例表(td_ontology_case)中创建实例抽取业务记录。待处理文档上传至 MinIO，由 rag-server 服务从 MinIO 接入并登记文件；随后按抽取要求完成资料准备。业务层发起 rag-ontology 服务实例抽取，传递本体对象、文档和模型配置，并保存服务任务标识。

后台任务依据对象定义调用模型，抽取具体实例、属性与关系。业务层查询实例任务状态；任务完成后获取三元组或结构化结果，转换为用户可以理解的实体、标签和关联关系。若界面需要超图等进一步加工结果，应跟踪其独立处理状态，不能把实例抽取成功直接当成所有图谱加工均完成。

4.2.3 各阶段的交接与控制

阶段	主要交接	业务放行条件
本体登记与 TTL 上传	业务本体记录关联文件标识。	文件上传成功且对应本体明确。

阶段	主要交接	业务放行条件
本体解析	业务本体关联服务本体及后台任务。	解析与对象结构生成完成。
实例准备	实例业务记录关联本体和待抽取文档。	本体可用、文档可访问、配置有效。
实例执行	实例业务记录关联服务抽取任务。	任务达到可获取结果的完成状态。
结果展示	业务层读取并转换结构化结果。	实体、属性、关系可正常查询与展示。

4.2.4 失败恢复与变更处理

TTL 结构错误时返回本体创建环节修正；本体未完成时阻止实例抽取；MinIO 访问失败时重试文档接入；模型调用失败时按任务策略重试。仅结果查询失败时先恢复查询，避免重复抽取。本体修改后应形成明确版本，并决定哪些实例重新抽取；旧任务继续保留其所使用的本体和配置引用。

4.3 文档入库与检索问答流程



图 5 文档入库与检索问答流程图

图 5 文档入库与检索问答流程图展示了文档知识从上传入库到在线问答的处理闭环。在文档入库阶段，文件由业务层和 rag-server 服务登记后，通过 RocketMQ 异步分发给 rag-loader 服务进行解析、OCR 和切分，再由 rag-embedding 服务生成向量并写入 Elasticsearch，索引校验完成后文档即可对外提供检索服务。在线问答阶段，用户请求经认证、网关和 Talkflow 编排后，在授权知识范围内执行文本或向量召回、重排序及模型生成，并返回答案及关联资料；对于对象属性、实体关系等结构化问题，可按需联合 rag-ontology 服务的本体检索结果增强回答。

4.3.1 文档入库

用户选择知识范围并上传资料，由 Java 业务层和 rag-server 服务登记文件及处理任务，原文保存到共享文件或对象存储。rag-server 服务通过 RocketMQ

提交解析任务；rag-loader 服务识别格式并提取内容，扫描件或复杂 PDF 由 rag-magicpdf 服务完成 OCR 和版面解析。切片保留文件及内容位置引用，便于后续检索展示。

切片准备完成后，通过 RocketMQ 进入异步嵌入环节。rag-embedding 服务调用 embedding_service 生成向量，将可检索文本、向量和必要的范围字段写入 Elasticsearch。完成索引验证后，业务层将文档标记为可检索。FAQ、摘要和关键词等衍生知识按需要生成，不要求所有任务同时完成才开放基础文本检索。

4.3.2 在线问答

用户问题通过认证和网关进入 Talkflow，业务层确定用户可访问的知识范围。检索服务在该范围内召回候选，rag-rerank 服务对候选进行相关性重排序；对话编排选择相关内容，控制上下文长度，再通过模型调用能力生成回答。答案与关联资料返回页面，便于用户核验。

对象属性比较或关系查询场景,可按需调用 rag-ontology 服务的结构化检索，并将实例属性和关系与文本检索结果共同组织到上下文。该调用使用已有本体与实例结果，不在每次问答中重新创建本体。启用此能力前，需确认任务范围、输出格式和来源展示契约。

4.3.3 异常和完成条件

阶段	完成条件	失败处理
文件接入与解析	原文可访问，解析切片生成。	保留原文与失败阶段，修复后重跑。

阶段	完成条件	失败处理
嵌入与索引	索引写入完成且可查询。	重试索引处理，不提前标记可检索。
召回与重排序	候选符合用户权限和相关性要求。	无相关内容时提示，限制无依据回答。
模型生成	请求正常完成，答案可展示。	超时或限额时返回提示，按策略降级。

5 模块职责与协作关系

模块	主要责任	上下游协作
Web 与 Gateway	页面入口、请求路由和统一接入。	将业务请求交给认证与业务服务。
Auth 与 Platform	登录身份及基础平台能力。	为业务范围校验和服务接入提供支撑。
Java 业务管理	业务记录、配置、任务编排及结果展示。	组织 RAG 和本体调用，维护业务与服务标识。
Talkflow 与 LLM-Proxy	对话步骤、上下文及模型调用组织。	按场景查询知识，再调用 LLM。

模块	主要责任	上下游协作
rag-server	文件登记、知识业务接口及处理触发。	衔接 MinIO、解析和嵌入流水线。
rag-loader 与 rag-magicpdf	格式解析、OCR 与文档切分。	生成切片供向量化和知识处理使用。
rag-embedding	内容索引、向量化调度和检索召回。	调用嵌入与重排模型，维护 ES 索引。
embedding_service 与 rag-rerank	向量计算与候选相关性评分。	为检索服务返回模型计算结果。
maas-ontology	类型与属性抽取、领域本体生成能力。	接收文档和场景配置，返回任务及产物。
rag-ontology Client 与 Worker	本体解析管理、实例抽取和结构化检索。	API 接收任务，Celery Worker 执行处理。